Comprehensive API Document for ArchAngel AML

---

1. Entity Matching

Purpose
Match entities (e.g., persons, companies) against a dataset using specific criteria such as name, date of birth, nationality, and other unique identifiers. This endpoint facilitates compliance, fraud detection, and verification processes.

Endpoint
`/api/match/`

HTTP Method
`POST`

Features

- Accepts entity details as JSON input, including schema type and properties.
- Matches entities against the specified dataset.
- Returns matching results with confidence scores based on matching criteria.

Input Example

json

```
{
  "queries": {
    "entity1": {
      "schema": "Person",
      "properties": {
        "name": ["John Doe"],
        "birthDate": ["1975-04-21"],
        "nationality": ["US"]
      }
    },
    "entity2": {
      "schema": "Company",
      "properties": {
```

```
        "name": ["Amazing Tech Ltd"],
        "jurisdiction": ["UK"],
        "registrationNumber": ["12345678"]
      }
    }
  }
}
```

Output Example

json

```
{
  "responses": {
    "entity1": {
      "query": { /* original input */ },
      "results": [
        {"id": "1001", "score": 0.95, "name": "John Doe"}
      ]
    },
    "entity2": {
      "query": { /* original input */ },
      "results": [
        {"id": "2001", "score": 0.88, "name": "Amazing Tech Ltd"}
      ]
    }
  }
}
```

---

2. Text-Based Search

Purpose
Perform free-text queries to search for entities in a dataset. This endpoint is useful for user-facing search interfaces.

Endpoint
`/api/search/`

HTTP Method
`GET`

Features

- Supports wildcard (`*`) and fuzzy searches for flexible querying.
- Provides filters for entity type, country, and topics.
- Returns a list of matching entities sorted by relevance.

Input Parameters

- `q`: Query string (required).
- `limit`: Number of results to return (default: 10).

Output Example

json

```
{
  "limit": 10,
  "results": [
    {"id": "1001", "name": "John Doe", "relevance": 0.92},
    {"id": "1002", "name": "Jane Doe", "relevance": 0.88}
  ]
}
```

---

3. Entity Fetching

Purpose
Fetch detailed information for a specific entity using its unique identifier.

Endpoint
`/api/entities/{entity_id}`

HTTP Method
`GET`

Features

- Retrieves all associated data for an entity.
- Includes adjacent entities, such as family members, subsidiaries, or addresses.

Output Example

json

```
{
  "id": "1001",
  "name": "John Doe",
  "attributes": {
    "birthDate": "1975-04-21",
    "nationality": "US",
    "address": "123 Main Street, NY"
  },
  "linkedEntities": [
    {"id": "2001", "type": "FamilyMember", "name": "Jane Doe"}
  ]
}
```

---

4. Reconciliation Services

Purpose
Reconcile local data with external datasets for data cleaning, validation, or augmentation.

Endpoint
`/api/reconcile/{dataset}`

HTTP Method
`GET`

Features

- Supports batch reconciliation operations.
- Returns scores and detailed reconciliation results for each entity.

Output Example

json

```
{
  "reconciled_entities": [
    {"id": "1001", "name": "John Doe", "matched_score": 0.95}
  ]
```

```
}
```

---

5. Sample API Schema

Example OpenAPI Specification (YAML)

yaml

```yaml
openapi: 3.0.0
info:
  title: Entity Management API
  description: API for entity matching, text-based search, entity
fetching, and reconciliation services.
  version: 1.0.0
servers:
  - url: https://api.angefin.com/
paths:
  /api/match/{dataset}:
    post:
      summary: Entity Matching
      parameters:
        - name: dataset
          in: path
          required: true
          schema:
            type: string
          description: Dataset name to query.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                queries:
                  type: object
                  additionalProperties:
                    type: object
```

```
                  properties:
                    schema:
                      type: string
                      example: "Person"
                    properties:
                      type: object
                      additionalProperties:
                        type: array
                        items:
                          type: string
      responses:
        '200':
          description: Matching results
          content:
            application/json:
              schema:
                type: object
```

---

6. Sample Code

FastAPI Implementation

python

```python
from fastapi import FastAPI, Path, Query
from typing import Dict

app = FastAPI()

@app.post("/api/match/{dataset}")
async def match_entities(dataset: str, queries: Dict[str, Dict]):
    results = {
        entity_id: {
            "query": details,
            "results": [{"id": "123", "score": 0.95}]
        } for entity_id, details in queries.items()
    }
    return {"responses": results}
```

```
@app.get("/api/search/{dataset}")
async def search_entities(dataset: str, q: str = Query(...), limit:
int = Query(10)):
    return {
        "limit": limit,
        "results": [{"id": "123", "name": "John Doe", "relevance":
0.9}]
    }

@app.get("/api/entities/{entity_id}")
async def fetch_entity(entity_id: str):
    return {
        "id": entity_id,
        "name": "John Doe",
        "attributes": {"birthDate": "1975-04-21", "nationality": "US"}
    }

@app.get("/api/reconcile/{dataset}")
async def reconcile_entities(dataset: str):
    return {
        "reconciled_entities": [
            {"id": "123", "name": "John Doe", "matched_score": 0.95}
        ]
    }
```

---

7. API Gateway Compliance

- **HTTPS**: All communication must use HTTPS for secure data transfer.
- **Documentation**: Use tools like **Redoc** or **Swagger UI** for API documentation.
- **Authentication**: API keys or OAuth for access control.
- **Scalability**: Design to handle large datasets and high concurrent requests.

AML Risk Scoring API Requirements

Overview

The **AML Risk Scoring API** is designed to compute risk scores for individuals or entities based on data provided or extracted through other systems, such as batch uploads or OCR-extracted information. This API integrates multiple risk factors, datasets, and algorithms to assign a score that determines the likelihood of involvement in money laundering, terrorism financing, or other financial crimes.

Core Functionality

Endpoints

1. **Submit Risk Scoring Request**: `/api/aml/risk/score`
2. **Fetch Risk Score Details**: `/api/aml/risk/details/{entity_id}`

Features

1. **Dynamic Risk Scoring**:
    - Compute risk scores using multiple factors such as:
        - Sanctions list matches.
        - Politically Exposed Persons (PEP) data.
        - Geographic risk (country-specific risk levels).
        - Adverse media coverage.
        - Transaction volume and frequency (if available).
    - Score range: `0.0` (low risk) to `1.0` (high risk).
2. **Customizable Risk Models**:
    - Allow dynamic configuration of weightage for each risk factor:
        - Example: Sanctions = 40%, Adverse Media = 30%, Geography = 20%, PEP = 10%.
3. **Result Categorization**:
    - Categorize entities based on risk score:
        - `0.0 - 0.3`: Low Risk
        - `0.4 - 0.6`: Medium Risk
        - `0.7 - 1.0`: High Risk
    - Return reasons for the assigned risk score.
4. **Real-Time and Batch Processing**:
    - Process single entities in real-time.
    - Option for batch risk scoring for multiple entities using pre-uploaded datasets.
5. **Detailed Reporting**:
    - Provide a breakdown of the score for transparency:
        - Factor contribution.
        - Matching datasets.
        - Relevant flags or reasons.

Risk Scoring Logic

Factors and Weightages

The risk score is calculated based on weighted contributions from the following factors:

Risk Factor	Description	Example	Default Weight (%)
Sanctions Match	Entity appears on sanctions lists (e.g., OFAC, UN, EU).	High match on OFAC list.	40%
Politically Exposed	Entity is flagged as a PEP.	Politician or family member of a politician.	10%
Geographic Risk	Entity operates in high-risk regions (based on FATF or similar standards).	Located in high-risk jurisdiction.	20%
Adverse Media	Entity has been mentioned negatively in news or adverse reports.	Report of fraud in media.	20%
Transaction Risk	Abnormal transaction patterns or large transaction volumes (if applicable).	Sudden large transfer.	10%

API Endpoints

1. Submit Risk Scoring Request

Endpoint
`POST /api/aml/risk/score`

Input Parameters

- **Entity Details** (required):
  - `name`: Entity name.
  - `date_of_birth`: Date of birth (if individual).
  - `nationality`: Nationality.
  - `address`: Residential or operational address.
  - `id_number`: Passport, national ID, or company registration number.

- **Transaction Details** (optional):
  - `transaction_volume`: Amount of transactions in a specific period.
  - `transaction_frequency`: Number of transactions in a specific period.

Input Example

json

```
{
  "entity": {
    "name": "John Doe",
    "date_of_birth": "1975-04-21",
    "nationality": "US",
    "id_number": "A12345678",
    "address": "123 Main Street, New York, USA"
  },
  "transactions": {
    "volume": 50000,
    "frequency": 5
  }
}
```

Response

json

```
{
  "entity_id": "1001",
  "risk_score": 0.87,
  "risk_category": "High Risk",
  "reasons": [
    "High confidence match on OFAC sanctions list.",
    "Located in high-risk jurisdiction (Country: XYZ).",
    "Adverse media reports found (e.g., fraud accusations)."
  ],
  "breakdown": {
    "Sanctions Match": 0.4,
    "PEP": 0.1,
    "Geography": 0.2,
    "Adverse Media": 0.15,
```

```
    "Transactions": 0.02
  }
}
```

---

2. Fetch Risk Score Details

Endpoint
`GET /api/aml/risk/details/{entity_id}`

Input Parameters

- `entity_id`: Unique identifier for the entity (required).

Output Example

json

```
{
  "entity_id": "1001",
  "name": "John Doe",
  "risk_score": 0.87,
  "risk_category": "High Risk",
  "details": {
    "sanctions": {
      "status": "Flagged",
      "datasets": ["OFAC", "EU"],
      "confidence": 0.95
    },
    "pep": {
      "status": "Not Flagged"
    },
    "geography": {
      "status": "High Risk",
      "country": "XYZ",
      "reason": "Listed as high-risk by FATF."
    },
    "adverse_media": {
      "status": "Flagged",
      "reports": [
```

```
        {"source": "News Article", "details": "Fraud accusation in
2023."}
      ]
    }
  }
}
```

Advanced Features

Custom Risk Models

- Allow clients to define custom weightages for risk factors during the API call:

json

```
{
  "entity": { /* entity details */ },
  "transactions": { /* transaction details */ },
  "weights": {
    "Sanctions Match": 50,
    "PEP": 10,
    "Geography": 20,
    "Adverse Media": 10,
    "Transactions": 10
  }
}
```

Batch Processing

- Support bulk risk scoring for multiple entities through `/api/aml/risk/score/batch`.

Input Example

json

```
{
  "batch": [
    {
      "name": "John Doe",
```

```
      "date_of_birth": "1975-04-21",
      "id_number": "A12345678"
    },
    {
      "name": "Acme Corporation",
      "nationality": "UK",
      "registration_number": "XYZ987654"
    }
  ]
}
```

Response

json

```
{
  "batch_id": "7890",
  "results": [
    {"entity_id": "1001", "risk_score": 0.87, "risk_category": "High
Risk"},
    {"entity_id": "1002", "risk_score": 0.25, "risk_category": "Low
Risk"}
  ]
}
```

Error Handling

- `400 Bad Request`: Invalid or incomplete input.
- `404 Not Found`: Entity ID not found for risk details.
- `500 Internal Server Error`: Processing failure or system error.

Authentication & Security

- Use API keys or OAuth tokens for secure access.
- Encrypt sensitive data in transit using HTTPS.
- Log and monitor all API activity for compliance and auditing.

Monitoring and Scaling

- Implement Prometheus and Grafana for monitoring API performance.
- Use scalable infrastructure (e.g., Kubernetes, AWS Lambda) for high-volume processing.

Batch Bulk Processing API for AML

Purpose

Enable the processing of large datasets containing entity information (e.g., names, addresses, dates of birth, company details) against sanctions lists, politically exposed persons (PEPs), and other AML datasets.

Endpoints

- **Upload Batch Data**: `/api/aml/batch/upload`
- **Process Batch**: `/api/aml/batch/process`
- **Retrieve Results**: `/api/aml/batch/results/{batch_id}`

Features

1. **Data Upload**:
    - Accept CSV, Excel, or JSON files containing entity information.
    - Validate the file structure (mandatory fields such as name, date of birth, or company registration number).
    - Support encryption during upload for sensitive data.
    - Return a `batch_id` for tracking processing status.
2. **Batch Processing**:
    - Match entities in the batch against AML datasets, such as:
        - Sanctions lists (OFAC, UN, EU, etc.).
        - Politically Exposed Persons (PEPs).
        - Adverse media checks.
    - Implement fuzzy matching for names, dates, and other attributes to reduce false negatives.
    - Provide a scoring mechanism (0-1) to rank the likelihood of matches.
    - Generate a detailed report, including:
        - Entity information.
        - Match status.
        - Risk score.
        - Associated datasets and reasons for flagging.
3. **Results Retrieval**:
    - Allow retrieval of processed results via API or downloadable files (CSV or JSON).

- Support filtering (e.g., only show flagged entities).

Sample API Workflows

1. **Uploading Batch Data**

Request:
json

```
POST /api/aml/batch/upload
Content-Type: multipart/form-data

{
  "file": "batch_entities.csv"
}
```

-

Response:
json

```
{
  "batch_id": "12345",
  "status": "uploaded",
  "message": "File uploaded successfully."
}
```

-

2. **Processing the Batch**

Request:
json

```
POST /api/aml/batch/process
Content-Type: application/json

{
  "batch_id": "12345"
}
```

-

Response:
json

```
{
  "batch_id": "12345",
  "status": "processing",
  "message": "Batch is being processed. Check back later for results."
}
```

-

3. **Retrieving Results**

Request:
json

```
GET /api/aml/batch/results/12345
```

-

Response:
json

```
{
  "batch_id": "12345",
  "status": "completed",
  "results": [
    {
      "entity": "John Doe",
      "date_of_birth": "1975-04-21",
      "match_status": "flagged",
      "risk_score": 0.92,
      "datasets": ["OFAC", "PEP"]
    },
    {
      "entity": "Acme Corp",
      "match_status": "clear",
      "risk_score": 0.1
    }
  ]
}
```

-

---

2. OCR-to-AML Lookup

Purpose

Enable the extraction of information from scanned documents (e.g., passports, national IDs) and perform AML lookups to identify sanctions, PEPs, or other risk factors.

Endpoints

- **Upload Document**: `/api/ocr/upload`
- **Extract Data**: `/api/ocr/extract`
- **AML Lookup**: `/api/ocr/lookup`

Features

1. **Document Upload**:
    - Accept images (JPEG, PNG) or PDFs.
    - Validate file size and format.
    - Use encryption for secure transmission.
2. **Data Extraction**:
    - Extract key information using Optical Character Recognition (OCR), including:
        - Name.
        - Date of Birth.
        - Passport or ID number.
        - Nationality.
    - Perform basic validations (e.g., valid date format, alphanumeric passport numbers).
    - Return extracted information for review.
3. **AML Lookup**:
    - Use extracted data to query AML datasets.
    - Support fuzzy matching to handle OCR inaccuracies.
    - Return a detailed match report, including:
        - Match status.
        - Risk score.
        - Associated datasets.

Sample API Workflows

1. **Uploading a Document**

Request:
json

```
POST /api/ocr/upload
Content-Type: multipart/form-data

{
  "file": "passport_image.jpg"
}
```

-

Response:
json

```
{
  "document_id": "67890",
  "status": "uploaded",
  "message": "Document uploaded successfully."
}
```

-

2. **Extracting Data**

Request:
json

```
POST /api/ocr/extract
Content-Type: application/json

{
  "document_id": "67890"
}
```

-

Response:
json

```
{
  "document_id": "67890",
  "status": "extracted",
  "data": {
    "name": "John Doe",
```

```
    "date_of_birth": "1975-04-21",
    "passport_number": "A12345678",
    "nationality": "US"
  }
}
```

-

3. **Performing AML Lookup**

Request:
json

```
POST /api/ocr/lookup
Content-Type: application/json

{
  "data": {
    "name": "John Doe",
    "date_of_birth": "1975-04-21",
    "passport_number": "A12345678",
    "nationality": "US"
  }
}
```

-

Response:
json

```
{
  "status": "flagged",
  "risk_score": 0.87,
  "datasets": ["OFAC", "PEP"],
  "reasons": [
    "Name matches with high confidence.",
    "Birthdate matches."
  ]
}
```

-

Common Features

Error Handling

- Return appropriate HTTP status codes:
  - `200`: Successful request.
  - `400`: Invalid input (e.g., malformed file, missing parameters).
  - `500`: Internal server error.
- Provide detailed error messages for debugging.

Authentication

- Use API keys or OAuth tokens to secure endpoints.

Rate Limiting

- Implement rate limiting to prevent abuse (e.g., 1000 requests per hour).

Scalability

- Support asynchronous processing for large datasets or high volumes of document uploads.
- Use message queues (e.g., RabbitMQ, Kafka) for batch processing.

Monitoring and Logging

- Monitor API performance using tools like Prometheus and Grafana.
- Log all requests and processing results for auditability.

AML Risk Scoring API Requirements

Overview

The **AML Risk Scoring API** is designed to compute risk scores for individuals or entities based on data provided or extracted through other systems, such as batch uploads or OCR-extracted information. This API integrates multiple risk factors, datasets, and algorithms to assign a score that determines the likelihood of involvement in money laundering, terrorism financing, or other financial crimes.

Core Functionality

Endpoints

1. **Submit Risk Scoring Request**: `/api/aml/risk/score`
2. **Fetch Risk Score Details**: `/api/aml/risk/details/{entity_id}`

Features

1. **Dynamic Risk Scoring**:
    - Compute risk scores using multiple factors such as:
        - Sanctions list matches.
        - Politically Exposed Persons (PEP) data.
        - Geographic risk (country-specific risk levels).
        - Adverse media coverage.
        - Transaction volume and frequency (if available).
    - Score range: `0.0` (low risk) to `1.0` (high risk).
2. **Customizable Risk Models**:
    - Allow dynamic configuration of weightage for each risk factor:
        - Example: Sanctions = 40%, Adverse Media = 30%, Geography = 20%, PEP = 10%.
3. **Result Categorization**:
    - Categorize entities based on risk score:
        - `0.0 - 0.3`: Low Risk
        - `0.4 - 0.6`: Medium Risk
        - `0.7 - 1.0`: High Risk
    - Return reasons for the assigned risk score.
4. **Real-Time and Batch Processing**:
    - Process single entities in real-time.
    - Option for batch risk scoring for multiple entities using pre-uploaded datasets.
5. **Detailed Reporting**:
    - Provide a breakdown of the score for transparency:
        - Factor contribution.
        - Matching datasets.
        - Relevant flags or reasons.

Risk Scoring Logic

Factors and Weightages

The risk score is calculated based on weighted contributions from the following factors:

Risk Factor	Description	Example	Default Weight (%)
Sanctions Match	Entity appears on sanctions lists (e.g., OFAC, UN, EU).	High match on OFAC list.	40%
Politically Exposed	Entity is flagged as a PEP.	Politician or family member of a politician.	10%
Geographic Risk	Entity operates in high-risk regions (based on FATF or similar standards).	Located in high-risk jurisdiction.	20%
Adverse Media	Entity has been mentioned negatively in news or adverse reports.	Report of fraud in media.	20%
Transaction Risk	Abnormal transaction patterns or large transaction volumes (if applicable).	Sudden large transfer.	10%

API Endpoints

1. Submit Risk Scoring Request

Endpoint
`POST /api/aml/risk/score`

Input Parameters

- **Entity Details** (required):
  - `name`: Entity name.
  - `date_of_birth`: Date of birth (if individual).
  - `nationality`: Nationality.
  - `address`: Residential or operational address.
  - `id_number`: Passport, national ID, or company registration number.
- **Transaction Details** (optional):
  - `transaction_volume`: Amount of transactions in a specific period.
  - `transaction_frequency`: Number of transactions in a specific period.

Input Example

json

```
{
  "entity": {
    "name": "John Doe",
    "date_of_birth": "1975-04-21",
    "nationality": "US",
    "id_number": "A12345678",
    "address": "123 Main Street, New York, USA"
  },
  "transactions": {
    "volume": 50000,
    "frequency": 5
  }
}
```

Response

json

```
{
  "entity_id": "1001",
  "risk_score": 0.87,
  "risk_category": "High Risk",
  "reasons": [
    "High confidence match on OFAC sanctions list.",
    "Located in high-risk jurisdiction (Country: XYZ).",
    "Adverse media reports found (e.g., fraud accusations)."
  ],
  "breakdown": {
    "Sanctions Match": 0.4,
    "PEP": 0.1,
    "Geography": 0.2,
    "Adverse Media": 0.15,
    "Transactions": 0.02
  }
}
```

---

2. Fetch Risk Score Details

Endpoint

`GET /api/aml/risk/details/{entity_id}`

Input Parameters

- `entity_id`: Unique identifier for the entity (required).

Output Example

json

```
{
  "entity_id": "1001",
  "name": "John Doe",
  "risk_score": 0.87,
  "risk_category": "High Risk",
  "details": {
    "sanctions": {
      "status": "Flagged",
      "datasets": ["OFAC", "EU"],
      "confidence": 0.95
    },
    "pep": {
      "status": "Not Flagged"
    },
    "geography": {
      "status": "High Risk",
      "country": "XYZ",
      "reason": "Listed as high-risk by FATF."
    },
    "adverse_media": {
      "status": "Flagged",
      "reports": [
        {"source": "News Article", "details": "Fraud accusation in
2023."}
      ]
    }
  }
}
```

Advanced Features

Custom Risk Models

- Allow clients to define custom weightages for risk factors during the API call:

json

```
{
  "entity": { /* entity details */ },
  "transactions": { /* transaction details */ },
  "weights": {
    "Sanctions Match": 50,
    "PEP": 10,
    "Geography": 20,
    "Adverse Media": 10,
    "Transactions": 10
  }
}
```

Batch Processing

- Support bulk risk scoring for multiple entities through `/api/aml/risk/score/batch`.

Input Example

json

```
{
  "batch": [
    {
      "name": "John Doe",
      "date_of_birth": "1975-04-21",
      "id_number": "A12345678"
    },
    {
      "name": "Acme Corporation",
      "nationality": "UK",
      "registration_number": "XYZ987654"
```

```
    }
  ]
}
```

Response

json

```
{
  "batch_id": "7890",
  "results": [
    {"entity_id": "1001", "risk_score": 0.87, "risk_category": "High
Risk"},
    {"entity_id": "1002", "risk_score": 0.25, "risk_category": "Low
Risk"}
  ]
}
```

Error Handling

- `400 Bad Request`: Invalid or incomplete input.
- `404 Not Found`: Entity ID not found for risk details.
- `500 Internal Server Error`: Processing failure or system error.

Authentication & Security

- Use JWT for secure access.
- Encrypt sensitive data in transit using HTTPS.
- Log and monitor all API activity for compliance and auditing.

Monitoring

- Implement Sentry for monitoring

Authentication and User Management

1. POST /auth/jwt/create/

1. **Endpoint Description:** Creates a JSON Web Token (JWT) for authentication.
2. **Endpoint:** `POST /auth/jwt/create/`
3. **Endpoint Parameter:** None (credentials in request body)
4. **Endpoint Request:**

{

"username": "string",

"password": "string"

}

1. **Endpoint Response:**

{

"access": "string",

"refresh": "string"

}

1. **Endpoint Request and Response Example:**
2. Request:

{

"username": "testuser",

"password": "password123"

}

1. Response:

{

"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMjMsImV4cCI6MTY3ODg4MDAwMH0.some_access_token_signature",

"refresh":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMjMsImV4cCI6MTY3ODg4MDAwMH0.some_refresh_token_signature"

}

2. POST /auth/jwt/refresh/

1. **Endpoint Description:** Refreshes an expired JWT access token using a refresh token.
2. **Endpoint:** `POST /auth/jwt/refresh/`
3. **Endpoint Parameter:** None (refresh token in request body)
4. **Endpoint Request:**

{

"refresh": "string"

}

1. **Endpoint Response:**

{

"access": "string"

}

1. **Endpoint Request and Response Example:**
2. Request:

{

"refresh":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMjMsImV4cCI6MTY3ODg4MDAwMH0.some_refresh_token_signature"

}

1. Response:

{

"access":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMjMsImV4cCI6MTY3ODg4MDAwMH0.some_new_access_token_signature"

}

3. POST /auth/jwt/verify/

1. **Endpoint Description:** Verifies the validity of a JWT access token.
2. **Endpoint:** `POST /auth/jwt/verify/`
3. **Endpoint Parameter:** None (token in request body)
4. **Endpoint Request:**

{

"token": "string"

}

1. **Endpoint Response:** (200 OK if valid, 401 Unauthorized if invalid)
2. Success: Empty response (200 OK)
3. Failure: Error details (401 Unauthorized)
4. **Endpoint Request and Response Example:**
5. Request:

{

"token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMjMsImV4cCI6MTY3ODg4MDAwMH0.some_access_token_signature"

}

1. Response (Success): (Empty response)
2. Response (Failure):

{

"detail": "Token is invalid or expired."

}

4. GET /auth/users/

1. **Endpoint Description:** Retrieves a list of users. (Requires authentication and appropriate permissions)
2. **Endpoint:** `GET /auth/users/`
3. **Endpoint Parameter:** Query parameters for filtering and pagination (e.g., `page`, `page_size`).
4. **Endpoint Request:** (Requires authentication in headers)
5. **Endpoint Response:**

[

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

},

// ... more users

]

1. **Endpoint Request and Response Example:**
2. Request: `GET /auth/users/?page=1&page_size=10` (with authentication headers)
3. Response:

[

{"id": 1, "username": "user1", "email": "[email address removed]", "first_name": "John", "last_name": "Doe", "is_active": true},

{"id": 2, "username": "user2", "email": "[email address removed]", "first_name": "Jane", "last_name": "Smith", "is_active": true}

]

5. POST /auth/users/

1. **Endpoint Description:** Creates a new user.
2. **Endpoint:** `POST /auth/users/`
3. **Endpoint Parameter:** None (user data in request body)
4. **Endpoint Request:**

{

"username": "string",

"password": "string",

"email": "string",

"first_name": "string",

"last_name": "string"

}

1. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": false

}

1. **Endpoint Request and Response Example:**
2. Request:

{

"username": "newuser",

"password": "securepassword",

"email": "[email address removed]",

"first_name": "New",

"last_name": "User"

}

1. Response:

{

"id": 123,

"username": "newuser",

"email": "[email address removed]",

"first_name": "New",

"last_name": "User",

"is_active": false

}

6. GET /auth/users/{id}/

1. **Endpoint Description:** Retrieves a specific user by ID. (Requires authentication and appropriate permissions)
2. **Endpoint:** `GET /auth/users/{id}/`
3. **Endpoint Parameter:** `id` (user ID) in the URL path.
4. **Endpoint Request:** (Requires authentication in headers)
5. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `GET /auth/users/123/` (with authentication headers)
3. Response:

{

"id": 123,

"username": "existinguser",

"email": "[email address removed]",

"first_name": "Existing",

"last_name": "User",

"is_active": true

}

7. PUT /auth/users/{id}/

1. **Endpoint Description:** Updates a specific user by ID (full update). (Requires authentication and appropriate permissions)
2. **Endpoint:** `PUT /auth/users/{id}/`
3. **Endpoint Parameter:** `id` (user ID) in the URL path.
4. **Endpoint Request:**

{

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `PUT /auth/users/123/` (with authentication headers)

{

"username": "updateduser",

"email": "[email address removed]",

"first_name": "Updated",

"last_name": "User",

"is_active": true

}

1. Response:

{

"id": 123,

"username": "updateduser",

"email": "[email address removed]",

"first_name": "Updated",

"last_name": "User",

"is_active": true

}

8. PATCH /auth/users/{id}/

1. **Endpoint Description:** Partially updates a specific user by ID. (Requires authentication and appropriate permissions)
2. **Endpoint:** `PATCH /auth/users/{id}/`
3. **Endpoint Parameter:** `id` (user ID) in the URL path.

4. **Endpoint Request:** (Partial user data in request body)

{

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `PATCH /auth/users/123/` (with authentication headers)

{

"first_name": "PartiallyUpdated"

}

1. Response:

{

"id": 123,

"username": "existinguser",

"email": "[email address removed]",

"first_name": "PartiallyUpdated",

"last_name": "User",

"is_active": true

}

9. DELETE /auth/users/{id}/

1. **Endpoint Description:** Deletes a specific user by ID. (Requires authentication and appropriate permissions)
2. **Endpoint:** `DELETE /auth/users/{id}/`
3. **Endpoint Parameter:** `id` (user ID) in the URL path.
4. **Endpoint Request:** (Requires authentication in headers)
5. **Endpoint Response:** (204 No Content on success)
6. **Endpoint Request and Response Example:**
7. Request: `DELETE /auth/users/123/` (with authentication headers)
8. Response: (204 No Content)

10. POST /auth/users/activation/

1. **Endpoint Description:** Activates a user account using an activation token.
2. **Endpoint:** `POST /auth/users/activation/`
3. **Endpoint Parameter:** None (activation token in request body)
4. **Endpoint Request:**

{

"uid": "string",

"token": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"uid": "some_uid",

"token": "some_activation_token"

}

1. Response: (204 No Content)

11. GET /auth/users/me/

1. **Endpoint Description:** Retrieves the currently authenticated user's information. (Requires authentication)
2. **Endpoint:** `GET /auth/users/me/`
3. **Endpoint Parameter:** None.
4. **Endpoint Request:** (Requires authentication in headers)
5. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `GET /auth/users/me/` (with authentication headers)
3. Response:

{

"id": 123,

"username": "currentuser",

"email": "[email address removed]",

"first_name": "Current",

"last_name": "User",

"is_active": true

}

12. PUT /auth/users/me/

1. **Endpoint Description:** Updates the currently authenticated user's information (full update). (Requires authentication)
2. **Endpoint:** `PUT /auth/users/me/`
3. **Endpoint Parameter:** None.
4. **Endpoint Request:**

{

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `PUT /auth/users/me/` (with authentication headers)

{

"username": "updatedcurrentuser",

"email": "[email address removed]",

"first_name": "UpdatedCurrent",

"last_name": "User",

"is_active": true

}

1. Response:

{

"id": 123,

"username": "updatedcurrentuser",

"email": "[email address removed]",

"first_name": "UpdatedCurrent",

"last_name": "User",

"is_active": true

}

13. PATCH /auth/users/me/

1. **Endpoint Description:** Partially updates the currently authenticated user's information. (Requires authentication)
2. **Endpoint:** `PATCH /auth/users/me/`
3. **Endpoint Parameter:** None.
4. **Endpoint Request:** (Partial user data in request body)

{

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Response:**

{

"id": 1,

"username": "string",

"email": "string",

"first_name": "string",

"last_name": "string",

"is_active": true

}

1. **Endpoint Request and Response Example:**
2. Request: `PATCH /auth/users/me/` (with authentication headers)

{

"first_name": "PartiallyUpdatedCurrent"

}

1. Response:

{

"id": 123,

"username": "currentuser",

"email": "[email address removed]",

"first_name": "PartiallyUpdatedCurrent",

"last_name": "User",

"is_active": true

}

14. DELETE /auth/users/me/

1. **Endpoint Description:** Deletes the currently authenticated user's account. (Requires authentication)
2. **Endpoint:** `DELETE /auth/users/me/`
3. **Endpoint Parameter:** None.
4. **Endpoint Request:** (Requires authentication in headers)
5. **Endpoint Response:** (204 No Content on success)
6. **Endpoint Request and Response Example:**
7. Request: `DELETE /auth/users/me/` (with authentication headers)
8. Response: (204 No Content)

15. POST /auth/users/resend_activation/

1. **Endpoint Description:** Resends the activation email to a user.
2. **Endpoint:** `POST /auth/users/resend_activation/`
3. **Endpoint Parameter:** None (email in request body)
4. **Endpoint Request:**

{

"email": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"email": "[email address removed]"

}

1. Response: (204 No Content)

16. POST /auth/users/reset_password/

1. **Endpoint Description:** Initiates a password reset process by sending a reset password email.
2. **Endpoint:** `POST /auth/users/reset_password/`
3. **Endpoint Parameter:** None (email in request body)
4. **Endpoint Request:**

{

"email": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"email": "[email address removed]"

}

1. Response: (204 No Content)

17. POST /auth/users/reset_password_confirm/

1. **Endpoint Description:** Confirms a password reset using a reset password token.
2. **Endpoint:** `POST /auth/users/reset_password_confirm/`
3. **Endpoint Parameter:** None (token and new password in request body)
4. **Endpoint Request:**

{

"uid": "string",

"token": "string",

"new_password": "string",

"re_new_password": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"uid": "some_uid",

"token": "some_reset_token",

"new_password": "newsecurepassword",

"re_new_password": "newsecurepassword"

}

1. Response: (204 No Content)

18. POST /auth/users/reset_username/

1. **Endpoint Description:** Initiates a username reset process by sending a reset username email.
2. **Endpoint:** `POST /auth/users/reset_username/`
3. **Endpoint Parameter:** None (email in request body)
4. **Endpoint Request:**

{

"email": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"email": "[email address removed]"

}

1. Response: (204 No Content)

19. POST /auth/users/reset_username_confirm/

1. **Endpoint Description:** Confirms a username reset using a reset username token.
2. **Endpoint:** `POST /auth/users/reset_username_confirm/`
3. **Endpoint Parameter:** None (token and new username in request body)
4. **Endpoint Request:**

{

"uid": "string",

"token": "string",

"new_username": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"uid": "some_uid",

"token": "some_reset_token",

"new_username": "newusername"

}

1. Response: (204 No Content)

20. POST /auth/users/set_password/

1. **Endpoint Description:** Sets a new password for the currently authenticated user. (Requires authentication)
2. **Endpoint:** `POST /auth/users/set_password/`
3. **Endpoint Parameter:** None (old and new passwords in request body)
4. **Endpoint Request:**

{

"new_password": "string",

"re_new_password": "string",

"current_password": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"new_password": "newsecurepassword",

"re_new_password": "newsecurepassword",

"current_password": "oldsecurepassword"

}

1. Response: (204 No Content)

21. POST /auth/users/set_username/

1. **Endpoint Description:** Sets a new username for the currently authenticated user. (Requires authentication)
2. **Endpoint:** `POST /auth/users/set_username/`
3. **Endpoint Parameter:** None (new username and current password in request body)
4. **Endpoint Request:**

{

"new_username": "string",

"current_password": "string"

}

1. **Endpoint Response:** (204 No Content on success, or error details)
2. **Endpoint Request and Response Example:**
3. Request:

{

"new_username": "newusername",

"current_password": "currentsecurepassword"

}

1. Response: (204 No Content)